

Estructura de datos

Listas enlazadas

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

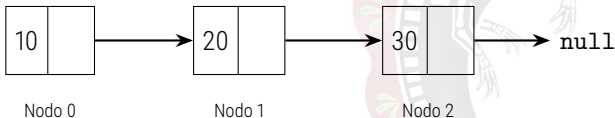
- 1 Listas enlazadas
- 2 Implementación
- 3 Recorrer una lista
- 4 Insertar nodos
- 5 Método obtener
- 6 Eliminar nodos
- 7 Buscar nodos
- 8 Limpiar lista



Listas enlazadas

Definición

Una **lista simplemente enlazada** almacena sus **elementos** en objetos llamados **nodos**, **conectados** mediante **referencias**.



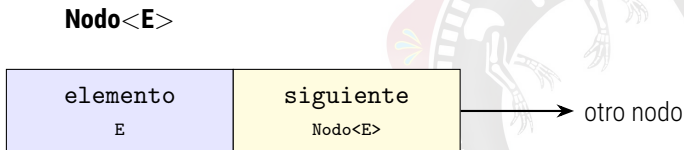
¡Importante!

Los nodos **no necesitan ocupar posiciones contiguas** en memoria. La relación entre ellos se establece mediante **referencias**.

Estructura de un nodo

Definición

Un **nodo** de una **lista simplemente enlazada** contiene **dos elementos**: el **elemento a almacenar** y la **referencia al siguiente nodo**.



elemento almacena el **dato asociado** al nodo.

siguiente almacena una **referencia** al **siguiente nodo**.

¡Importante!

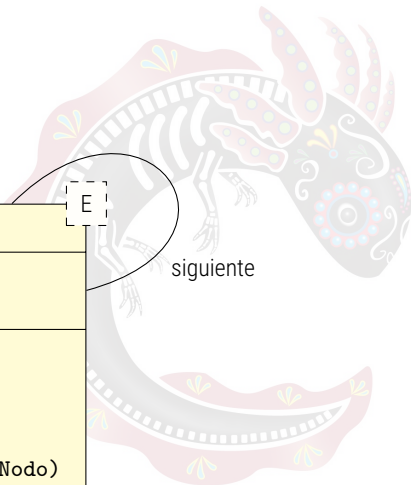
La clase **Nodo<E>** contiene una **referencia** a **otro objeto** de su **mismo tipo**.

Modelado de un Nodo

Nodo
-elemento : E -siguiente : Nodo
+Nodo(elemento : E) +getElemento() : E +setElemento(elemento : E) +getSiguiente() : Nodo +setSiguiente(siguiente : Nodo)

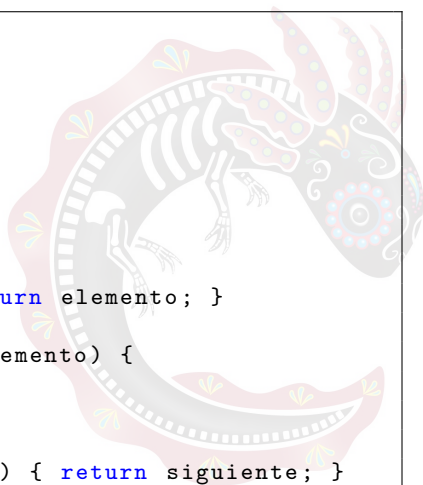
E

siguiente



Implementación clase `Nodo<E>`

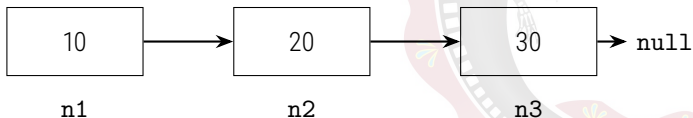
```
public class Nodo<E> {  
    private E elemento;  
    private Nodo<E> siguiente;  
  
    public Nodo(E elemento) {  
        this.elemento = elemento;  
        this.siguiente = null;  
    }  
  
    public E getElemento() { return elemento; }  
  
    public void setElemento(E elemento) {  
        this.elemento = elemento;  
    }  
  
    public Nodo<E> getSiguiente() { return siguiente; }  
  
    public void setSiguiente(Nodo<E> siguiente) {  
        this.siguiente = siguiente;  
    }  
}
```



Conexión entre nodos

```
Nodo<Integer> n1 = new Nodo<>(10);  
Nodo<Integer> n2 = new Nodo<>(20);  
Nodo<Integer> n3 = new Nodo<>(30);
```

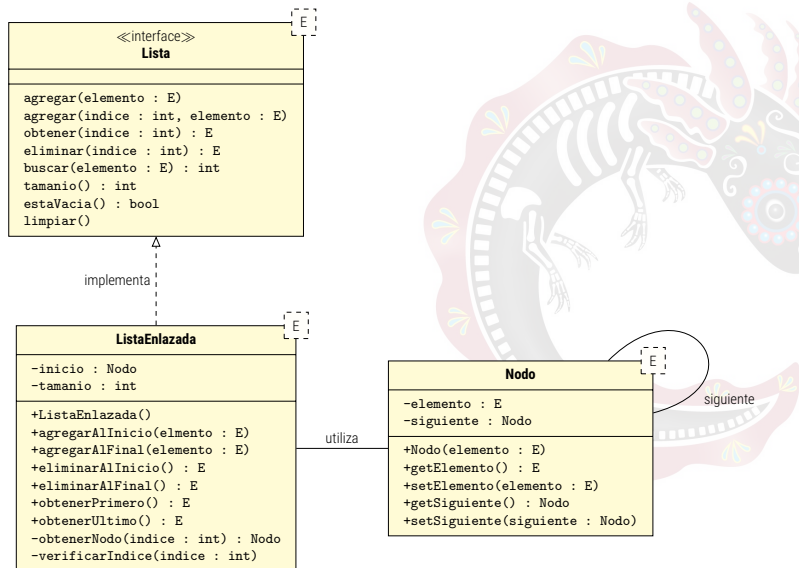
```
n1.setSiguiente(n2);  
n2.setSiguiente(n3);
```



Consecuencia

A partir de **n1** podemos **alcanzar** a **todos los nodos** siguiendo sucesivamente la **referencia** siguiente.

Diagrama de clases



Representación de ListaEnlazada

Definición

La lista **solamente** necesita **conservar** una **referencia** al **primer nodo**:



`tamano = 4`

Invariantes

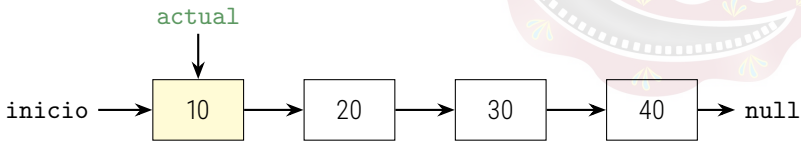
- Si la lista **está vacía**, `inicio == null`.
- El **último nodo** tiene `siguiente == null`.
- `tamano` indica el **número de nodos** de la lista.

Recorrido de la lista

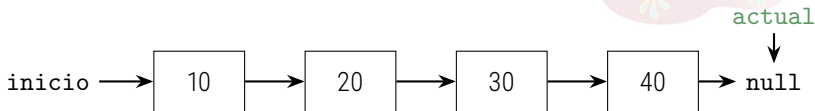
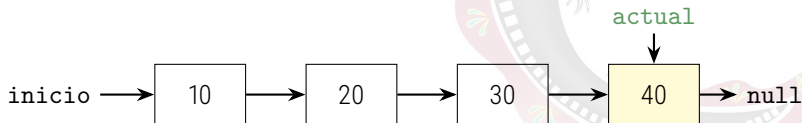
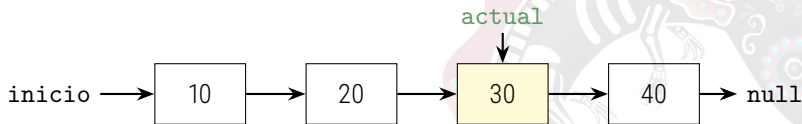
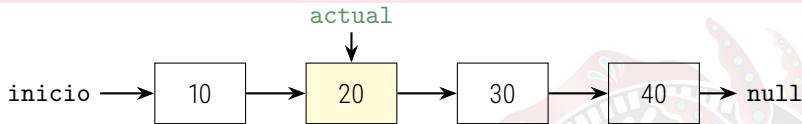
```
1: actual  $\leftarrow$  inicio
2: while actual  $\neq$  null do
3:   procesar actual.elemento
4:   actual  $\leftarrow$  actual.siguiente
5: end while
```

¡Importante!

No existe **acceso directo** por **índice**, para llegar a una **posición** debemos comenzar en **inicio** y **seguir las referencias**.



Recorrido de la lista

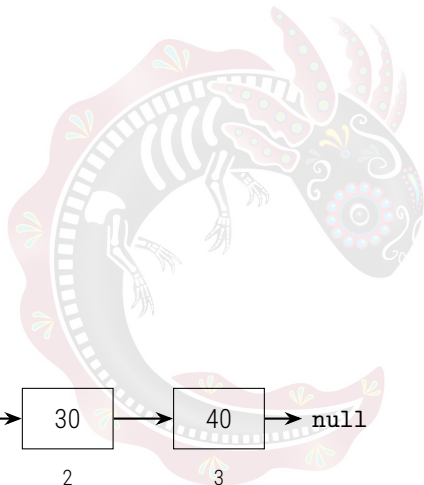
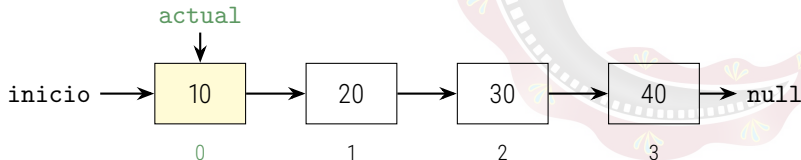


Método auxiliar obtenerNodo()

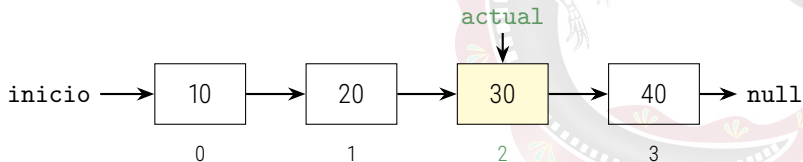
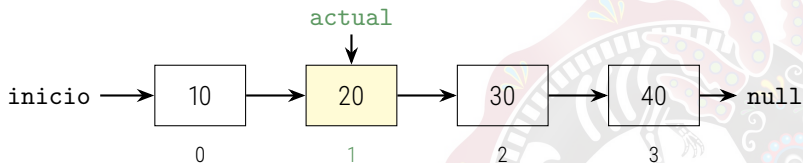
Require: indice

- 1: $actual \leftarrow inicio$
- 2: **for** $i \leftarrow 0$ **hasta** $indice - 1$ **do**
- 3: $actual \leftarrow actual.siguiente$
- 4: **end for**
- 5: **return** $actual$

Para obtener el nodo en la posición 2:



Método auxiliar obtenerNodo()



obtenerNodo(2) → 30

Agregar al final

Require: elemento

- 1: $nuevo \leftarrow \text{Nodo}(\text{elemento})$
- 2: **if** la lista está vacía **then**
- 3: $inicio \leftarrow nuevo$
- 4: **else**
- 5: $actual \leftarrow inicio$
- 6: **while** $actual.siguiete \neq \text{null}$ **do**
- 7: $actual \leftarrow actual.siguiete$
- 8: **end while**
- 9: $actual.siguiete \leftarrow nuevo$
- 10: **end if**
- 11: $tamano \leftarrow tamano + 1$



Agregar un nodo al final

Complejidad

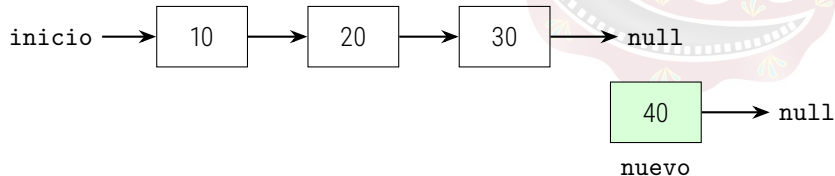
Con únicamente la referencia **inicio**, es necesario **recorrer toda la lista**

$$O(n)$$

Antes:

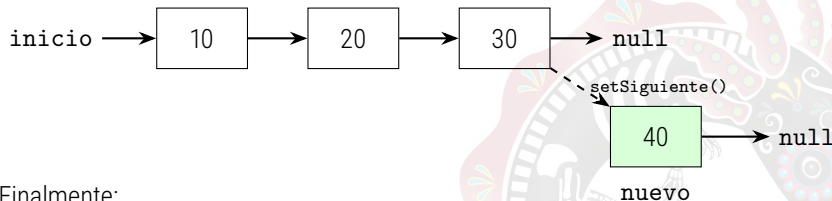


Creamos un nuevo nodo:

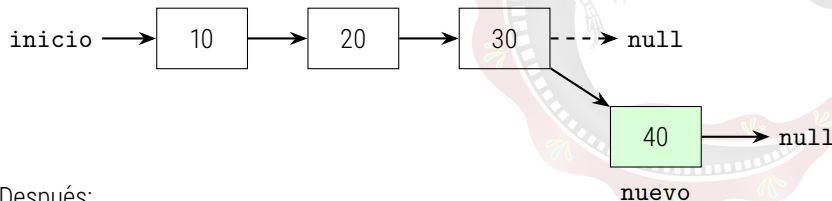


Agregar un nodo al final

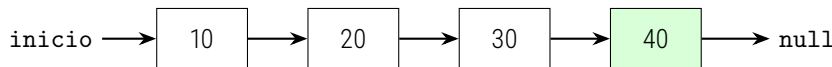
Enlazamos:



Finalmente:



Después:



Insertar al inicio

Para ejecutar:

agregar(0,5)

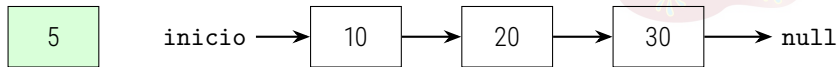
se requieren solamente dos cambios de referencias:

- 1: $nuevo \leftarrow Nodo(5)$
- 2: $nuevo.siguiete \leftarrow inicio$
- 3: $inicio \leftarrow nuevo$
- 4: $tamano \leftarrow tamano + 1$

Inicialmente:



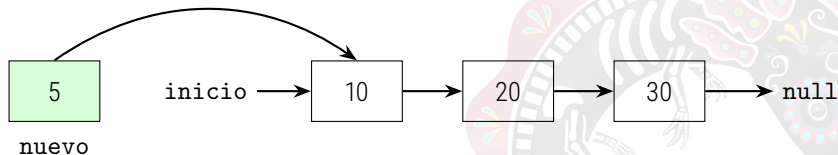
Creamos un nuevo nodo:



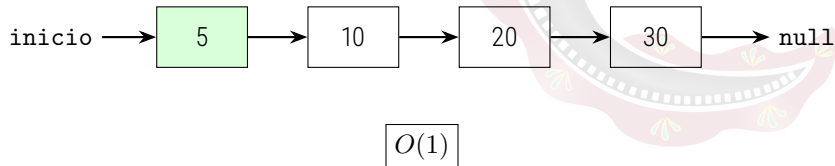
nuevo

Insertar al inicio

Enlazamos el nodo nuevo:



Cambiamos la referencia del inicio:



Insertión en una posición

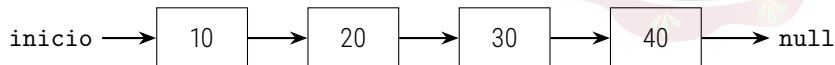
Insertar 25 en la posición 2:

- 1: $anterior \leftarrow obtenerNodo(indice - 1)$
- 2: $nuevo \leftarrow Nodo(elemento)$
- 3: $nuevo.siguiente \leftarrow anterior.siguiente$
- 4: $anterior.siguiente \leftarrow nuevo$
- 5: $tamanio \leftarrow tamanio + 1$

¡Importante!

Primero debe conservarse la referencia al nodo que originalmente seguía a **anterior**; después se modifica el enlace de **anterior**.

Inicialmente:



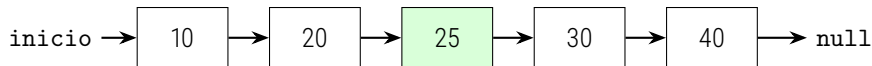
Inserción en una posición



nuevo



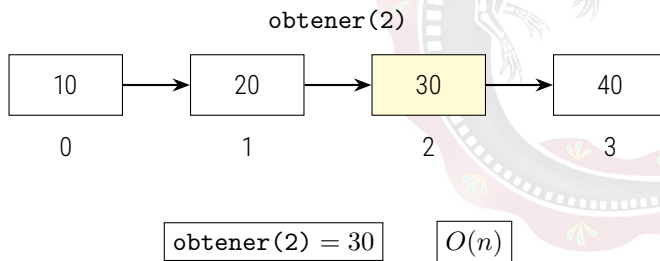
nuevo



Método obtener(indice)

Require: indice

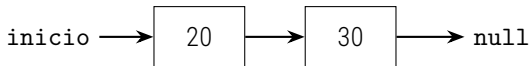
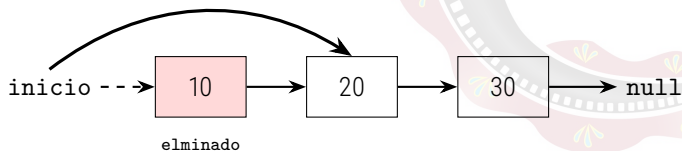
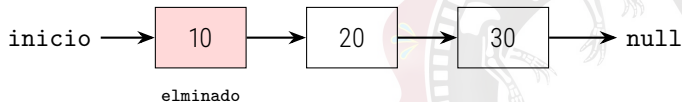
- 1: verificar que el índice sea válido
- 2: $nodo \leftarrow obtenerNodo(indice)$
- 3: **return** $nodo.elemento$



Eliminar el primer elemento

Si `indice == 0`:

- 1: $\text{eliminado} \leftarrow \text{inicio}$
- 2: $\text{inicio} \leftarrow \text{inicio.siguiente}$
- 3: $\text{tamanio} \leftarrow \text{tamanio} - 1$
- 4: **return** $\text{eliminado.elemento}$

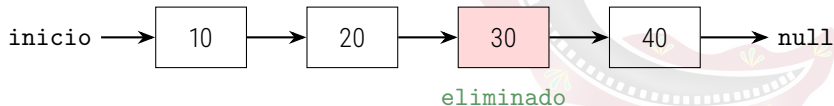


Eliminación de un nodo intermedio

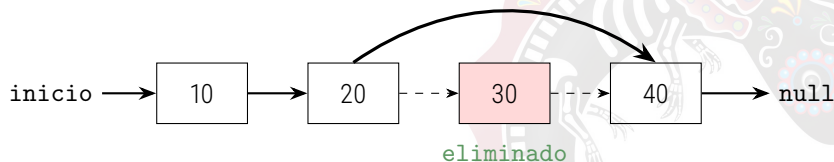
Eliminar el elemento de la posición 2:

- 1: $\text{anterior} \leftarrow \text{obtenerNodo}(\text{indice} - 1)$
- 2: $\text{eliminado} \leftarrow \text{anterior.siguiente}$
- 3: $\text{anterior.siguiente} \leftarrow \text{eliminado.siguiente}$
- 4: $\text{tamanio} \leftarrow \text{tamanio} - 1$
- 5: **return** $\text{eliminado.elemento}$

Inicialmente:



Eliminación de un nodo intermedio



Método buscar(elemento)

Require: elemento

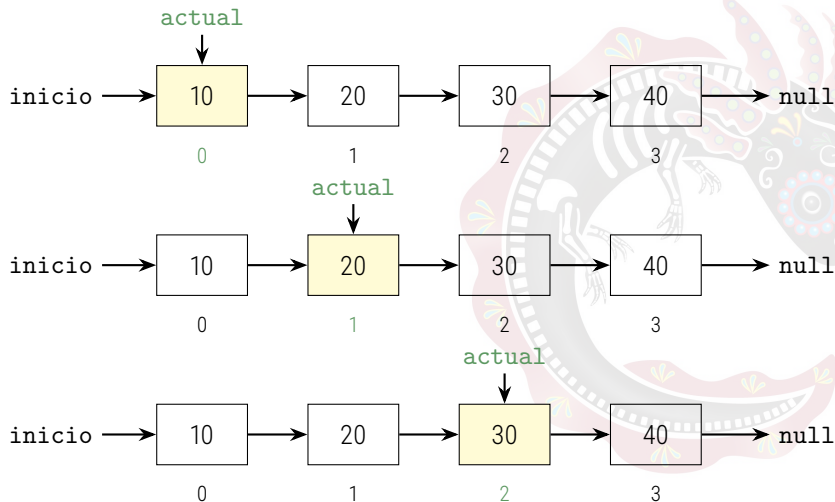
```

1: actual  $\leftarrow$  inicio
2: indice  $\leftarrow$  0
3: while actual  $\neq$  null do
4:   if actual.elemento = elemento then
5:     return indice
6:   end if
7:   actual  $\leftarrow$  actual.siguiente
8:   indice  $\leftarrow$  indice + 1
9: end while
10: return -1
    
```

buscar(30)



Método buscar(elemento)



return 2

Métodos tamaño() y estaVacia()

tamano()

```
return tamanio
```

```
inicio → [10] → [20] → [30] → null
```

```
tamano() = 3
```

estaVacía()

```
return tamanio = 0
```

inicio $\rightarrow$ null

true

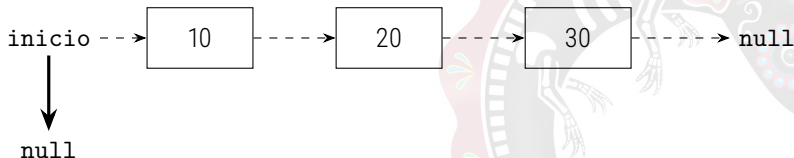
Complejidad

Ambos métodos tienen complejidad:

 $O(1)$

Método limpiar()

- 1: $inicio \leftarrow null$
- 2: $tamaño \leftarrow 0$



¿Qué sucede con los nodos?

Al **desaparecer la referencia** desde **inicio**, los nodos **dejan de ser accesibles** desde la lista y **pueden** ser **recuperados** posteriormente por el **recolector de basura** de Java.

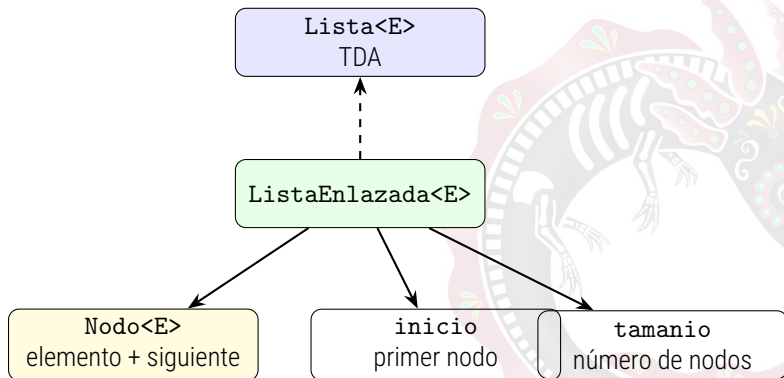
ListaArreglos **VS.** ListaEnlazada

Operación	ListaArreglos	ListaEnlazada
Obtener por índice	$O(1)$	$O(n)$
Buscar	$O(n)$	$O(n)$
Insertar al inicio	$O(n)$	$O(1)$
Insertar en posición	$O(n)$	$O(n)$
Eliminar al inicio	$O(n)$	$O(1)$
Tamaño	$O(1)$	$O(1)$

¡Importante!

No existe una implementación universalmente “mejor”, la elección **depende** de las **operaciones que se realizan con mayor frecuencia.**

Construcción de `ListaEnlazada<E>`



Idea fundamental

La interfaz `Lista<E>` establece **qué operaciones** deben existir;
`ListaEnlazada<E>` determina **cómo implementarlas** mediante objetos `Nodo<E>` conectados por referencias.

Fin de la presentación

¿Preguntas o comentarios?

